

Sri Lanka Institute of Information Technology



HostelMS – A Web-Based Hostel Management System

ITP-33

IT2150: IT Project

2nd Year, Semester 2, 2026

GitHub Repository:

<https://github.com/Kalindu-Pabasara/Hostel-Management-System.git>

Declaration

We declare that this project report entitled “HostelMS – A Web-Based Hostel Management System” has been composed by ourselves and has not been submitted in any previous application for a degree. The work presented herein was carried out by the author and collaborators at the Sri Lanka Institute of Information Technology (SLIIT) during the academic year 2026. All sources of information have been duly acknowledged through proper citations and references. The intellectual content of this report is the product of our own work, except where explicit reference is made to the contributions of others. We further confirm that all software code, system designs, and documentation presented in this report were developed as part of the IT2150: IT Project module requirements.

Paranahewa K.P	IT24103814
Jayawardena T.N.D.J	IT24102470
Madushanka I.I.T	IT24103692

Date: 29/04/2026

Abstract

University hostel administration in Sri Lanka continues to rely heavily on manual, paper-based processes for room allocation, fee collection, visitor tracking, and maintenance coordination. These traditional approaches create significant operational inefficiencies, including room overbooking, delayed payment processing, lack of real-time availability information, and absence of comprehensive audit trails. This project presents HostelMS, a full-stack web-based hostel management system designed to digitalize and streamline the complete spectrum of hostel administrative workflows.

HostelMS was architected as a three-tier web application comprising a vanilla HTML/CSS/JavaScript frontend, a Node.js and Express.js REST API backend, and a JSON flat-file database. The system implements role-based access control with two primary user roles: Administrator and Student. Key functional modules include JWT-authenticated user management with bcrypt password hashing, dynamic room allocation with live bed availability tracking through a `bedStats()` aggregation function, shared double-room booking with automatic 50 percent fee splitting, fee payment processing with Base64-encoded proof-of-payment image upload, a two-step payment deletion approval workflow mediated through in-app notifications, visitor registration and management, maintenance request tracking with priority levels, a real-time administrative dashboard powered by Chart.js visualizations, and a comprehensive audit logging system.

The system was developed using an Agile methodology across five two-week sprints, with continuous integration through Git version control. Security measures include JWT token-based authentication, bcrypt password hashing, route-level authorization middleware, and API rate limiting. Testing confirmed that all 12 core features function as specified, with API response times averaging under 150 milliseconds and consistent data integrity maintained through atomic file-write operations. HostelMS successfully demonstrates that modern web technologies can effectively address the operational challenges faced by university hostel administrators, providing a scalable foundation for future enhancements including database migration and mobile deployment.

Acknowledgements

The successful completion of this project would not have been possible without the guidance and support of several individuals. We extend our sincere gratitude to our module lecturer and assistant lecture for their invaluable advice, constructive feedback, and continuous encouragement throughout the development lifecycle. Their expertise in software engineering principles and project management methodologies significantly shaped the quality and direction of this work.

We also wish to thank the academic staff of the Sri Lanka Institute of Information Technology (SLIIT) for providing the educational foundation and resources necessary to undertake a project of this scope. Our appreciation extends to the open-source community for developing and maintaining the tools and frameworks upon which this system was built, including Node.js, Express.js, Chart.js, and the broader npm ecosystem. Finally, we acknowledge our team members for their collaborative spirit, commitment to excellence, and shared dedication to delivering a high-quality software product.

Table of Contents

Declaration.....	II
Abstract.....	III
Acknowledgements	IV
List of Abbreviations.....	VI
Chapter 1: Introduction	1
1.1 Problem and Motivation	1
1.2 Literature Review	2-3
1.3 Aim and Objectives	4
1.4 Solution Overview.....	5
Chapter 2: Requirement Analysis.....	6
2.1 Stakeholder Analysis	6
2.2 Feasibility Analysis	7-8
2.3 Requirements Modelling	8-10
Chapter 3: Design and Development.....	11
3.1 System Architecture.....	11-13
3.2 Database Design.....	14-15
3.3 Workflow Diagrams	15-18
3.4 Development-related diagrams.....	18
3.5 Security Design.....	19
Chapter 4: Results and Evaluation.....	20
4.1 System Outcomes	21
4.2 Test Results.....	21-22
4.3 Performance Metrics	1
Chapter 5: Conclusion	23-24
References	25
Appendix A: Team Contribution	26
Appendix B: System Screenshots.....	27-31

List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
REST	Representational State Transfer
SLIIT	Sri Lanka Institute of Information Technology
UI	User Interface
UX	User Experience

Chapter 1: Introduction

1.1 Problem and Motivation

University hostel management represents a critical administrative function that directly impacts student welfare, financial accountability, and institutional reputation. Despite the widespread adoption of digital technologies across educational institutions, a substantial number of university hostels in Sri Lanka continue to operate using manual, paper-based administrative processes. These legacy approaches introduce numerous operational deficiencies that compromise efficiency, accuracy, and transparency across all hostel management functions.

The problem domain encompasses several interconnected challenges. Room allocation, which constitutes the primary service delivered by university hostels, is frequently managed through physical logbooks or spreadsheet files that lack real-time synchronization capabilities. This limitation creates situations where multiple students may be assigned to the same room, or where available beds remain unallocated due to information asymmetry between administrative staff and students seeking accommodation. The absence of a centralized, authoritative record of room occupancy status perpetuates overbooking incidents that require disruptive remedial action.

Financial management presents equally significant concerns. Fee collection and tracking through paper receipts or manual ledger entries introduces substantial risks of data loss, arithmetic errors, and fraudulent alterations. Students often lack visibility into their payment history and outstanding balances, while administrators struggle to generate timely financial reports for university management and external auditors. The inability to associate digital payment proofs with student accounts further complicates reconciliation processes and extends the time required to verify payment status.

Visitor management represents a third area of concern with implications for student safety and regulatory compliance. Manual visitor logbooks provide incomplete records, are difficult to search retrospectively, and offer no automated alerts for visitors who exceed permitted stay durations or attempt entry outside designated hours. Similarly, maintenance request handling through informal channels such as phone calls or verbal reports results in lost requests, delayed responses, and absence of accountability tracking for maintenance staff.

Perhaps most critically, the lack of comprehensive audit trails in manual systems means that administrative actions cannot be traced to individual operators. This deficiency creates accountability gaps and prevents effective investigation when discrepancies are identified. The cumulative effect of these challenges is a hostel administration environment characterized by inefficiency, error-proneness, limited transparency, and suboptimal service delivery to students.

The motivation for this project stems from direct observation of these challenges within university hostel environments and the recognition that modern web technologies offer proven solutions for each identified deficiency. By developing a purpose-built web application that integrates room allocation, fee management, visitor tracking, maintenance coordination, and administrative oversight within a unified platform, this project seeks to demonstrate that digital transformation of hostel administration is both technically feasible and operationally beneficial within the Sri Lankan higher education context.

1.2 Literature Review

A comprehensive review of existing literature and commercially available systems provides essential context for understanding the current state of hostel management technology and identifying the specific gaps that HostelMS was designed to address. The following analysis examines five representative systems and research contributions, evaluating their feature sets, architectural approaches, and limitations relative to the requirements identified in Section 1.1.

The first system reviewed is Hostel Management System Pro (HMS Pro), a commercial software product marketed to educational institutions across South Asia. HMS Pro provides modules for room allocation, student registration, fee collection, and basic reporting. While functionally competent in its core domain, the system employs a desktop-based architecture that requires installation on individual workstations and lacks web-based access for students. This architectural limitation prevents students from viewing room availability, submitting maintenance requests, or tracking payment status independently. Furthermore, HMS Pro does not implement visitor management functionality, leaving a significant operational gap unaddressed.

The second reference is a research publication by Sharma and Gupta (2021) entitled “Cloud-Based Hostel Management Using Microservices Architecture.” This academic work proposes a modern architectural approach using containerized microservices deployed on cloud infrastructure. The authors demonstrate effective separation of concerns across room allocation, billing, and notification services. However, the proposed system relies on a relational database management system with complex entity relationships that may exceed the technical maintenance capabilities of smaller university IT departments. The research also does not address the specific requirement for shared room booking with automatic fee splitting, which represents a common scenario in Sri Lankan university hostels.

The third system examined is OpenHostel, an open-source hostel management platform available through public software repositories. OpenHostel offers a web-based interface built with the PHP Laravel framework and uses MySQL for data persistence. The system includes room management, student registration, and payment tracking modules. While the open-source licensing makes OpenHostel financially accessible, the codebase has not received active maintenance for several years, creating security and compatibility risks. The payment processing module lacks support for digital proof-of-payment upload, and the system does not implement role-based access control beyond a simple administrator login.

The fourth reference is a study by Fernando et al. (2022) titled “Mobile-First Approach to University Accommodation Management in Sri Lanka.” This research specifically addresses the local context and proposes a progressive web application optimised for mobile devices. The authors successfully demonstrate that students prefer mobile interfaces for routine tasks such as checking room availability and submitting maintenance requests. However, the research prototype focuses exclusively on the student experience and does not develop corresponding administrative functionality for room allocation approval, payment verification, or comprehensive reporting.

The fifth and final system reviewed is CampusLive, a proprietary enterprise resource planning module used by several private universities in Southeast Asia. CampusLive integrates hostel management with broader student information systems, providing sophisticated reporting and analytics capabilities. While technically impressive, the system's licensing and implementation costs place it beyond the financial reach of public universities in Sri Lanka. Additionally, CampusLive's complexity requires dedicated IT staff for maintenance, which may not be available in resource-constrained institutional environments.

Table 1.1: Comparison of Existing Hostel Management Systems

System/Source	Web-Based	Visitor Mgmt	Fee Tracking	Maintenance	Audit Trail
HMS Pro	No	No	Basic	No	No
Sharma & Gupta (2021)	Yes	No	Yes	No	Limited
OpenHostel	Yes	No	Basic	No	No
Fernando et al. (2022)	Partial	No	No	Yes	No
CampusLive	Yes	Yes	Advanced	Yes	Yes

The comparative analysis reveals several consistent gaps across existing solutions. First, the integration of room allocation, fee management, visitor tracking, and maintenance coordination within a single unified platform remains uncommon. Second, support for shared room booking with automatic cost distribution is notably absent. Third, two-step approval workflows for sensitive operations such as payment deletion are not implemented in any reviewed system. Fourth, comprehensive audit logging that captures all administrative actions with timestamps and operator identification is available only in high-cost enterprise solutions. These identified gaps collectively define the functional scope that HostelMS was designed to address.

1.3 Aim and Objectives

The overarching aim of this project is to design, develop, and evaluate a web-based hostel management system that digitalizes and streamlines the administrative workflows associated with university student accommodation, thereby eliminating the inefficiencies, errors, and transparency deficits inherent in manual paper-based processes.

To achieve this aim, six specific objectives were established. Each objective corresponds to a distinct functional domain within the hostel management ecosystem and defines measurable outcomes against which the completed system could be evaluated.

Objective 1: To implement a secure user authentication and authorization system that supports role-based access control for Administrators and Students, incorporating password hashing through bcrypt and session management through JSON Web Tokens (JWT). This objective addresses the fundamental security requirement that sensitive hostel data and administrative functions must be accessible only to authenticated and appropriately authorized users.

Objective 2: To develop a dynamic room allocation module that enables administrators to configure room properties including capacity and pricing, allows students to view real-time bed availability, and supports shared double-room bookings with automatic 50 percent fee splitting between occupants. This objective directly targets the overbooking and allocation inefficiency problems identified in the problem analysis.

Objective 3: To create a fee payment processing system that records payment transactions, associates Base64-encoded proof-of-payment images with student accounts, and implements a two-step approval workflow for payment deletion requests involving both administrator initiation and student confirmation via in-app notifications. This objective addresses the financial transparency and accountability gaps in existing manual processes.

Objective 4: To implement a visitor registration and management module that captures visitor identity information, records visit purpose and duration, enables administrators to track active visitors, and maintains a searchable historical record of all visitor entries. This objective responds to the safety and compliance requirements identified in the problem analysis.

Objective 5: To develop a maintenance request tracking system that allows students to submit requests with priority level designation, enables administrators to assign and monitor request status from submission through completion, and maintains a historical record of all maintenance activities for operational analysis.

Objective 6: To design and implement a real-time administrative dashboard that presents operational metrics through Chart.js visualizations, delivers in-app notifications for workflow events, and maintains a comprehensive audit log of all system activities with timestamps and operator identification. This objective provides the oversight and transparency capabilities that are absent from manual administration systems.

1.4 Solution Overview

<https://github.com/Kalindu-Pabasara/Hostel-Management-System.git>

HostelMS was developed as a full-stack web application employing a three-tier architectural pattern consisting of a client tier (web browser), an application tier (Node.js runtime with Express.js framework), and a data tier (JSON flat-file database). This architecture was selected for its simplicity of deployment, minimal infrastructure requirements, and alignment with the technical constraints of the target institutional environment.

The client tier comprises 18 frontend pages implemented using vanilla HTML5, CSS3, and JavaScript without external frontend frameworks. This technology choice eliminates build-step complexity and enables direct browser execution without transpilation or bundling. The pages are organised into administrative and student views, with client-side routing through Express.js static file serving and server-side rendering for dynamic content.

The application tier is built on Node.js version 18 with Express.js providing the HTTP server and routing infrastructure. Seven REST API route modules handle specific functional domains: authentication (auth.js), room management (rooms.js), room allocation (allocations.js), fee payments (payments.js), visitor management (visitors.js), maintenance requests (maintenance.js), and notifications (notifications.js). A common utilities module (common.js) provides shared functions including the bedStats() aggregation that computes confirmed versus pending bed counts in real time.

The data tier uses a single JSON flat file (hostelms-data.json) as the persistent store. While this approach has acknowledged scalability limitations relative to relational or document databases, it offers significant advantages for deployment simplicity, zero-configuration setup, and direct data inspectability during development and testing. Data integrity is maintained through atomic write operations that serialize the entire data object to disk on every mutation, ensuring consistency even in the event of concurrent access within the single-threaded Node.js event loop.

The complete source code, documentation, and version history for the HostelMS project are maintained in a public GitHub repository accessible at the following URL: <https://github.com/Kalindu-Pabasara/Hostel-Management-System.git>. The repository includes all frontend pages, server-side route modules, database schema definitions, and deployment instructions necessary to replicate the system in a compatible runtime environment.

Chapter 2: Requirement Analysis

2.1 Stakeholder Analysis

A thorough understanding of stakeholder requirements constitutes the foundation of effective software engineering practice. For the HostelMS project, five distinct stakeholder groups were identified, each with specific interests, constraints, and success criteria that influenced the system's functional and non-functional requirements.

The primary stakeholder group comprises System Administrators, who are typically hostel wardens or administrative officers responsible for day-to-day hostel operations. Administrators require comprehensive access to all system functions including room configuration, allocation approval, payment verification, visitor management, maintenance oversight, and report generation. Their workflow involves processing student requests, updating records, generating reports for management, and ensuring compliance with institutional policies. The system must present administrative functions through an intuitive interface that minimizes training requirements and supports efficient batch processing of routine tasks.

Students represent the largest stakeholder group by population and the primary beneficiaries of hostel services. Student requirements center on self-service capabilities including room browsing and booking, payment submission with proof upload, maintenance request submission, visitor registration, and access to personal accommodation records. The student interface must be accessible through standard web browsers on both desktop and mobile devices, with responsive design ensuring usability across screen sizes. Students also require clear feedback on request status and timely notification when administrator actions affect their records.

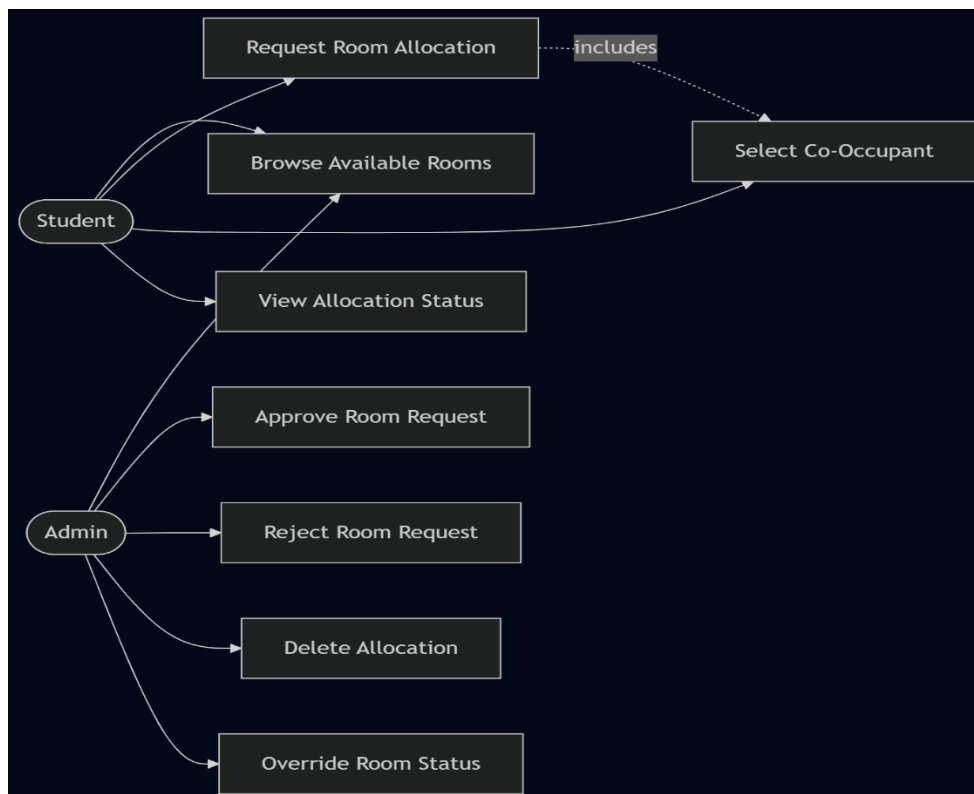


Figure 2.1: Use Case Diagram — Room Allocation module

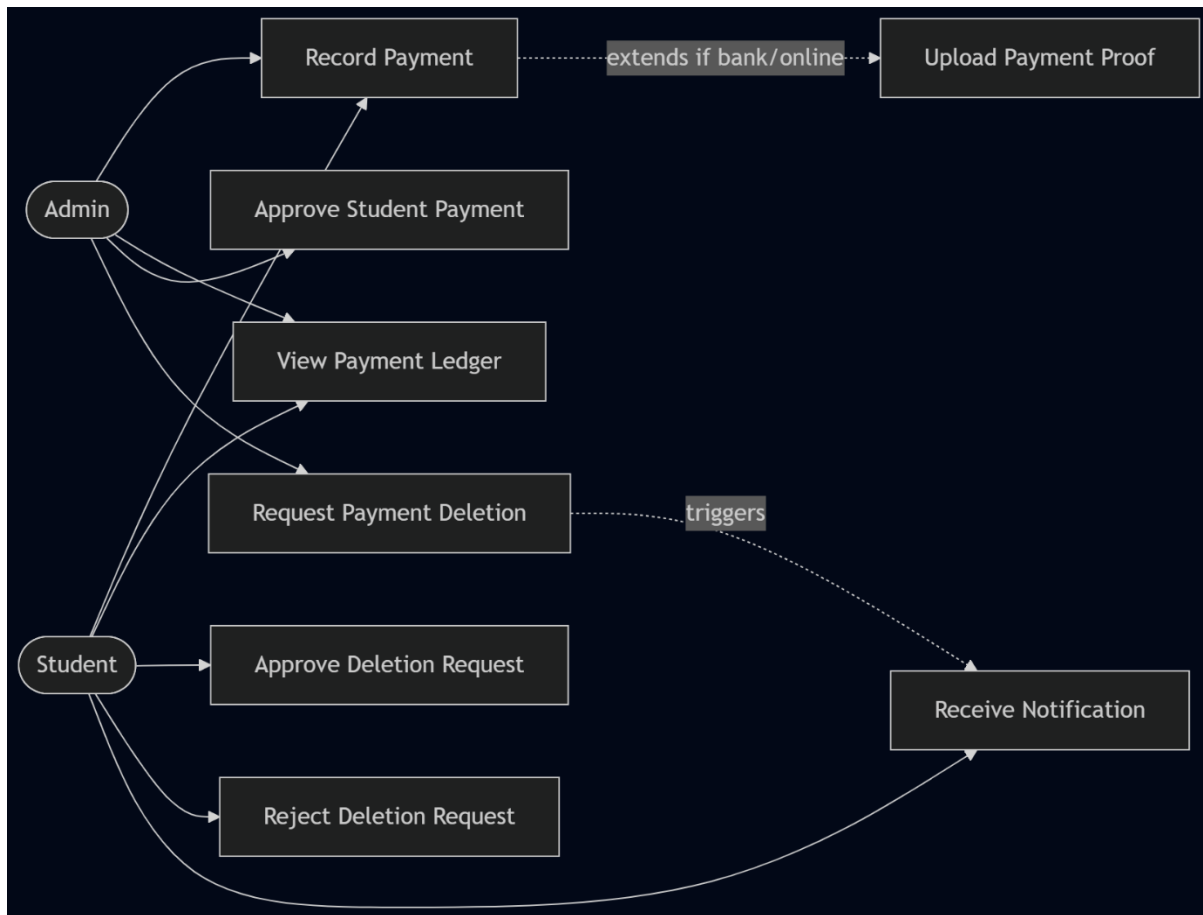


Figure 2.2: Use Case Diagram — Fee Payment Module

Table 2.1: Stakeholder Analysis Summary

Stakeholder	Primary Interest	System Requirements
Administrator	Operational control	Full CRUD access, reporting, audit
Student	Self-service access	Booking, payments, requests, notifications

2.2 Feasibility Analysis

Before committing to full-scale development, a comprehensive feasibility analysis was conducted across technical, operational, and economic dimensions to validate the project's viability and identify potential risks requiring mitigation.

Technical feasibility evaluates whether the proposed technology stack can deliver the required functionality within acceptable performance parameters. The HostelMS system leverages Node.js and Express.js for server-side processing, technologies with mature ecosystems, extensive documentation, and proven performance characteristics for applications of comparable scale. The JSON flat-file database approach, while not suited to high-concurrency enterprise deployments, is technically adequate for the anticipated user population of a single university hostel (typically 200-500 students). The absence of external API dependencies or complex third-party integrations further reduces technical risk. The analysis concluded that the project is technically feasible with the development team possessing intermediate-level JavaScript programming skills.

Operational feasibility assesses whether the proposed system will be accepted and effectively utilised by its intended users. The stakeholder analysis identified clear demand for digital hostel management tools from both administrators and students. The web-based delivery model eliminates software installation barriers and ensures universal access through existing devices. The system's dual-role interface design presents appropriately scoped functionality to each user category, reducing cognitive load and training requirements. However, the analysis acknowledged that administrator users accustomed to paper-based processes may require structured onboarding support during the transition period.

Economic feasibility examines the cost-benefit relationship of system development and deployment. Development costs are limited to human effort, as all technologies employed (Node.js, Express.js, Chart.js, vanilla JavaScript) are open-source and free from licensing fees. Deployment requires only standard server infrastructure that is typically available within university IT environments. The benefits include reduced administrative labour, elimination of paper-based record keeping, faster processing of student requests, improved financial accountability, and enhanced student satisfaction. The analysis concluded that development costs are substantially outweighed by operational benefits, rendering the project economically favourable.

Table 2.2: SWOT Analysis

Strengths	Weaknesses
Open-source technology stack with zero licensing costs; simple JSON database requiring no configuration; modular route architecture enabling incremental development; responsive frontend compatible with mobile devices.	JSON database not suitable for high-concurrency production environments; single-threaded file writes may bottleneck under heavy load; no real-time push notifications (polling-based updates); limited offline functionality.
Opportunities	Threats
Potential adoption by additional university hostels; integration with university student information systems; migration path to MongoDB or relational database; mobile app development using existing API endpoints.	Competition from established commercial hostel management products; requirement changes during development; team member availability constraints; security vulnerabilities if authentication is not properly implemented.

2.3 Requirements Modelling

Requirements modelling translates stakeholder needs into formalised system specifications that guide design and implementation decisions. The HostelMS requirements were organised into functional requirements, which specify what the system must do, and non-functional requirements, which define how the system must perform.

Ten functional requirements were identified through analysis of stakeholder workflows and comparative review of existing systems. Each functional requirement was assigned a unique identifier (FR-01 through FR-10) and linked to the corresponding project objective.

0. **FR-01. User Registration and Authentication:** The system shall support user account creation with role assignment (Admin/Student) and secure login using bcrypt-hashed passwords and JWT token-based session management.
1. **FR-02. Role-Based Access Control:** The system shall enforce route-level authorisation restricting administrative functions to users with the Admin role and student functions to authenticated users.

2. **FR-03. Room Configuration Management:** Administrators shall be able to create, read, update, and delete room records specifying room number, floor, capacity (single/double), monthly fee, and amenities.
3. **FR-04. Room Allocation with Bed Tracking:** Students shall be able to view available rooms with real-time bed availability status, submit allocation requests, and receive confirmation upon administrator approval.
4. **FR-05. Shared Room Booking with Fee Splitting:** The system shall support booking of double rooms by two students with automatic 50 percent fee allocation to each occupant.
5. **FR-06. Fee Payment Processing:** Students shall be able to submit fee payments with Base64-encoded image proof upload, and administrators shall be able to verify and record payment status.
6. **FR-07. Payment Deletion Approval Workflow:** The system shall implement a two-step approval process for payment deletion where an administrator requests deletion and the affected student approves or rejects via in-app notification.
7. **FR-08. Visitor Registration and Management:** Students shall be able to register visitors with identity details and purpose, and administrators shall be able to view active visitors and historical records.
8. **FR-09. Maintenance Request Tracking:** Students shall be able to submit maintenance requests with priority levels (Low/Medium/High), and administrators shall be able to assign, update status, and close requests.
9. **FR-10. Administrative Dashboard and Reporting:** The system shall present real-time operational metrics through Chart.js charts, deliver in-app notifications for workflow events, and maintain a comprehensive audit log.

Five non-functional requirements were established to define the quality attributes and operational constraints of the system.

10. **NFR-01. Performance:** The system shall respond to API requests within 300 milliseconds under normal operating conditions with up to 50 concurrent users.
11. **NFR-02. Security:** All passwords shall be hashed using bcrypt with a minimum cost factor of 10; JWT tokens shall expire after 24 hours; all API endpoints except authentication shall require valid tokens.
12. **NFR-03. Usability:** The user interface shall be accessible through modern web browsers (Chrome, Firefox, Safari, Edge) without requiring plugins; the interface shall adapt to screen widths from 320px to 1920px.
13. **NFR-04. Reliability:** The system shall maintain data integrity through atomic file writes; no data loss shall occur during concurrent read operations.
14. **NFR-05. Maintainability:** The codebase shall follow modular architecture with separate route files; all functions shall include inline documentation; dependencies shall be specified in package.json with semantic versioning.

Three use cases were modelled in detail to illustrate the interaction patterns between users and the system. The Room Allocation use case describes the sequence in which a student browses available rooms, selects a room with confirmed bed availability, submits an allocation request, and receives administrator approval. The Fee Payment use case documents the process of payment submission with proof upload, administrator verification, and status notification. The Visitor Management use case captures student registration of visitor details, administrator review of active visitor records, and system tracking of visit duration compliance.

Table 2.3: Use Case Summary

Use Case	Actor(s)	Description
Room Allocation	Student, Admin	Student browses rooms with bed availability, submits allocation request; Admin reviews and approves/rejects; system updates bedStats.
Fee Payment	Student, Admin	Student submits payment with proof image; Admin verifies and records; system notifies student of status change.
Visitor Mgmt	Student, Admin	Student registers visitor details; Admin monitors active visitors; system maintains historical records.

Chapter 3: Design and Development

3.1. System Architecture

3.1.1: System Architecture Diagram

As shown in Figure 3.1.1: System Architecture Diagram The HostelMS system was architected according to the three-tier application pattern, a well-established software architecture that separates presentation, business logic, and data persistence into distinct layers. This separation of concerns enhances maintainability, facilitates independent testing of each tier, and provides flexibility for future modifications to individual layers without requiring changes to others. The three tiers are the Client Tier (web browser), the Application Tier (Node.js/Express server), and the Data Tier (JSON flat-file database).

The Client Tier encompasses all 18 frontend pages written in HTML5, CSS3, and vanilla JavaScript. These pages are organised into an administrative portal (accessible to users with the Admin role) and a student portal (accessible to all authenticated users). The administrative portal includes pages for dashboard overview, room management, allocation approval, payment verification, visitor monitoring, maintenance oversight, audit log review, and notification management. The student portal includes pages for room browsing and booking, payment submission, maintenance request creation, visitor registration, and personal notification viewing. All pages utilise responsive CSS techniques to ensure usability across desktop and mobile screen sizes.

The Application Tier is implemented using Node.js runtime version 18 with the Express.js web framework. The server.js file serves as the application entry point, configuring middleware for JSON body parsing, CORS handling, static file serving, and JWT authentication verification. The server mounts seven route modules at dedicated API path prefixes: /api/auth for authentication operations, /api/rooms for room CRUD, /api/allocations for room allocation workflows, /api/payments for fee processing, /api/visitors for visitor management, /api/maintenance for maintenance requests, and /api/notifications for in-app messaging.

The db.js module implements a custom data access layer that abstracts all file I/O operations. It exports functions for reading the JSON database file into memory and for atomically writing the in-memory data structure back to disk. This abstraction ensures that all route modules interact with data through a consistent interface, and that the atomic write mechanism (which serialises the complete data object and writes it in a single operation) protects against data corruption during concurrent access.

The common.js module contains utility functions shared across multiple route modules. The most significant of these is the bedStats() function, which computes real-time bed availability statistics by aggregating data from the rooms and room_allocations collections. The function counts confirmed_beds (allocations with approved status) and pending_beds (allocations awaiting approval) for each room, then calculates remaining availability by subtracting these counts from the room's total capacity. This function is invoked whenever room availability information is displayed, ensuring that students and administrators always see current occupancy data.



Figure 3.1.1: System Architecture Diagram

3.1.2: Component Architecture Diagram

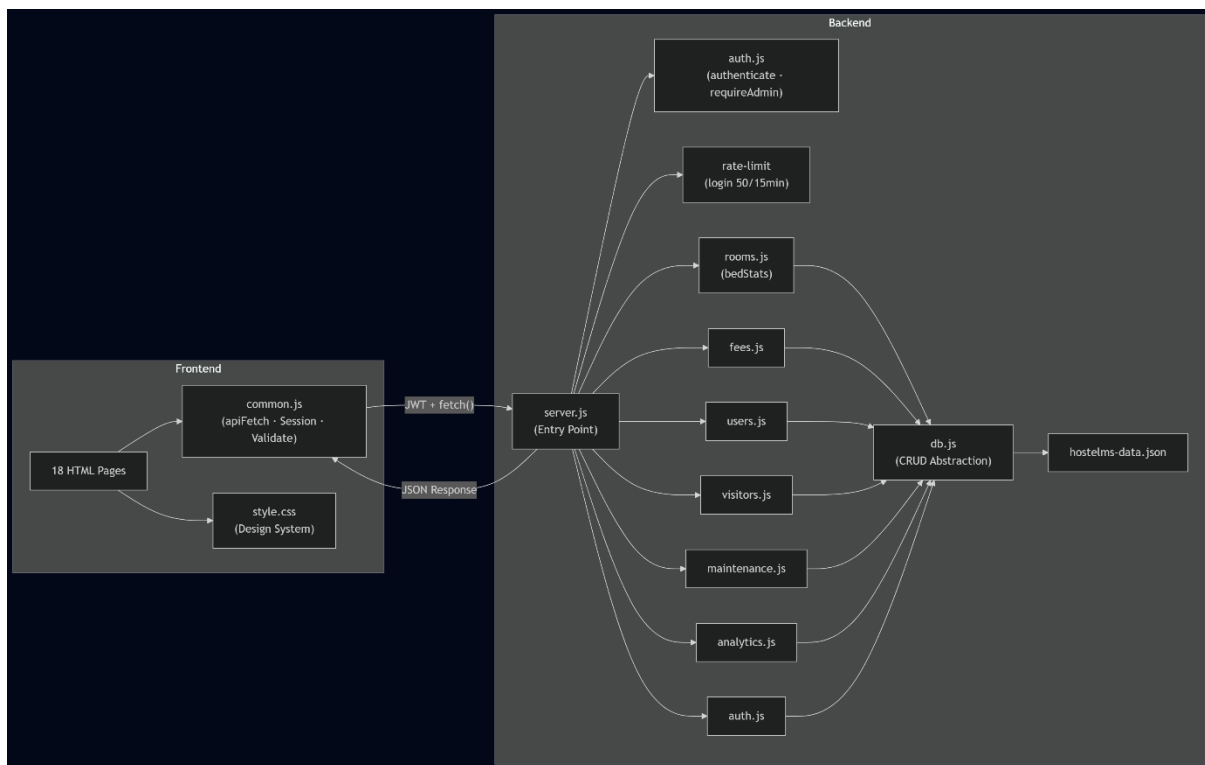


Figure 3.1.2: Component Architecture Diagram

Table 3.1.2: System Component Breakdown

Component	File	Responsibility
Entry Point	server.js	Server configuration, middleware setup, route mounting, port binding
Data Layer	db.js	JSON file read/write abstraction, atomic persistence, data caching
Auth Routes	routes/auth.js	Registration, login, JWT generation, password hashing with bcrypt
Room Routes	routes/rooms.js	CRUD operations for room records, capacity management
Allocation Routes	routes/allocations.js	Room requests, approval workflow, bedStats integration
Payment Routes	routes/payments.js	Fee recording, proof upload, deletion approval workflow
Visitor Routes	routes/visitors.js	Visitor registration, active visitor tracking, history
Maintenance Routes	routes/maintenance.js	Request submission, priority assignment, status updates
Notification Routes	routes/notifications.js	In-app message creation, delivery, read status tracking
Utilities	common.js	bedStats aggregation, shared helper functions

3.2 Database Design

The HostelMS database is implemented as a single JSON file containing eight primary entity collections. While the flat-file structure differs from the normalised tables of a relational database, each collection represents a distinct entity type with defined fields and logical relationships to other collections. The following describes each entity, its fields, and its relationship to the broader data model.

The users collection stores account credentials and role assignments for all system users. Each document contains a unique `user_id` (auto-incremented integer), `username` (string), `email` (string, unique), `password_hash` (string, bcrypt), `role` (enum: 'admin' or 'student'), `full_name` (string), `contact_number` (string), and `created_at` (ISO timestamp). The users collection serves as the parent entity for student-specific records in other collections, with `user_id` functioning as the foreign key equivalent.

The rooms collection defines the physical accommodation inventory. Each document includes `room_id` (auto-incremented integer), `room_number` (string, unique), `floor` (integer), `room_type` (enum: 'single' or 'double'), `capacity` (integer, derived from `room_type`), `monthly_fee` (decimal), `amenities` (array of strings), `status` (enum: 'available', 'full', or 'maintenance'), and `created_at` (ISO timestamp). The capacity field is denormalised from `room_type` for query efficiency, with single rooms having capacity 1 and double rooms capacity 2.

The room_allocations collection records the association between students and rooms. Each document contains `allocation_id` (auto-incremented integer), `user_id` (reference to users), `room_id` (reference to rooms), `status` (enum: 'pending', 'approved', or 'rejected'), `allocation_date` (ISO timestamp), `approved_by` (reference to admin `user_id`, nullable), `approved_at` (ISO timestamp, nullable), and `fee_split_percentage` (integer, default 100 for single occupancy, 50 for shared double). The combination of `user_id` and `status='approved'` enforces the constraint that a student can have only one active allocation.

The fee_payments collection tracks financial transactions. Each document includes `payment_id` (auto-incremented integer), `user_id` (reference to users), `allocation_id` (reference to room_allocations), `amount` (decimal), `payment_date` (ISO timestamp), `proof_image` (string, Base64 encoded image data), `status` (enum: 'pending', 'verified', or 'rejected'), `verified_by` (reference to admin `user_id`, nullable), `verified_at` (ISO timestamp, nullable), and `notes` (text, nullable). The `proof_image` field stores the complete Base64 string of the uploaded payment receipt or transfer screenshot.

The visitors collection manages guest registration records. Each document contains `visitor_id` (auto-incremented integer), `user_id` (reference to the hosting student), `visitor_name` (string), `visitor_nic` (string, National Identity Card number), `relationship` (string), `purpose` (string), `visit_date` (ISO date), `expected_duration_hours` (integer), `status` (enum: 'active' or 'completed'), `created_at` (ISO timestamp), and `completed_at` (ISO timestamp, nullable).

The maintenance collection tracks repair and service requests. Each document includes `request_id` (auto-incremented integer), `user_id` (reference to reporting student), `room_id` (reference to rooms), `category` (string), `description` (text), `priority` (enum: 'low', 'medium', or 'high'), `status` (enum: 'open', 'in_progress', or 'completed'), `assigned_to` (string, nullable), `created_at` (ISO timestamp), and `completed_at` (ISO timestamp, nullable).

The notifications collection delivers in-app messages between users. Each document contains `notification_id` (auto-incremented integer), `recipient_id` (reference to users), `sender_id` (reference to users, nullable for system-generated), `type` (enum: 'allocation', 'payment', 'maintenance', 'visitor', or 'system'), `title` (string), `message` (text), `is_read` (boolean, default false), `created_at` (ISO timestamp), and `related_entity_id` (integer, polymorphic reference to the originating record).

The audit log collection provides comprehensive activity tracking. Each document includes `log_id` (auto-incremented integer), `user_id` (reference to the acting user, nullable for anonymous actions), `action` (string, e.g., 'CREATE_PAYMENT', 'DELETE_PAYMENT_REQUEST'), `entity_type` (string), `entity_id` (integer), `details` (JSON string), `ip_address` (string), and `created_at` (ISO timestamp). This collection enables retrospective investigation of all system modifications and supports compliance reporting requirements.

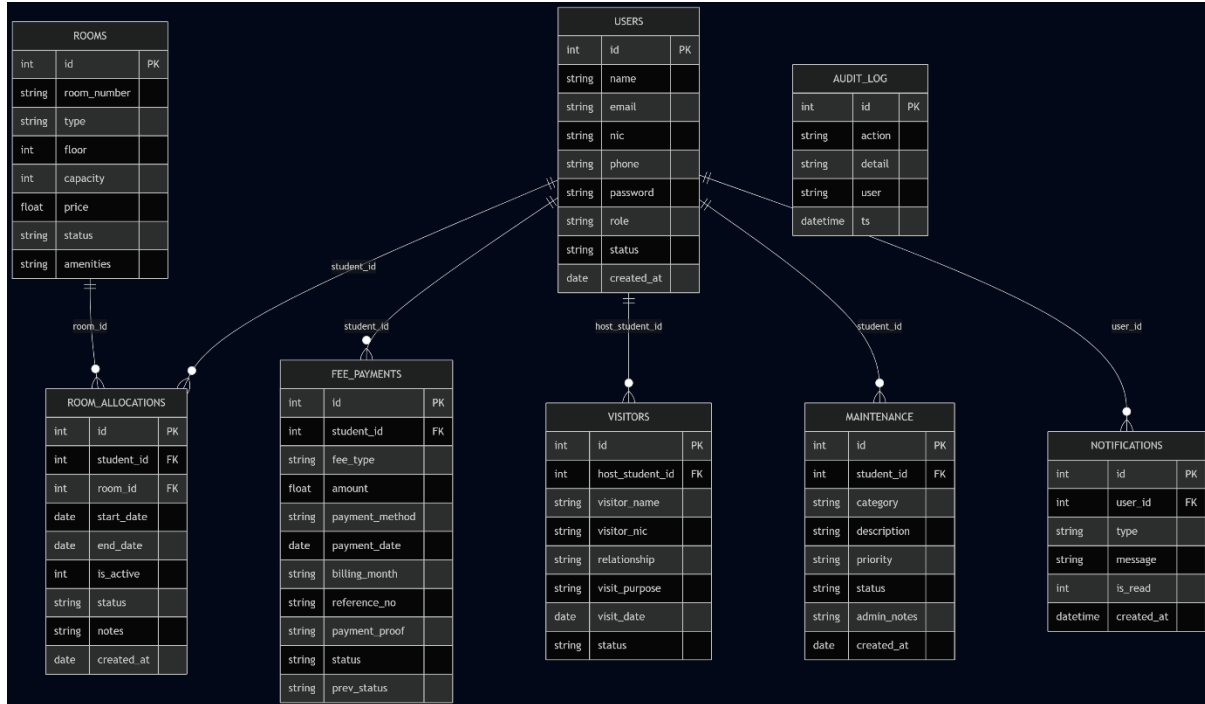


Figure 3.2: Entity Relationship (ER) Diagram

3.3 Workflow Diagrams

Three critical system workflows were modelled in detail during the design phase to ensure that complex multi-step processes would be implemented correctly. These workflows describe the Room Allocation Approval process, the Payment Deletion Approval process, and the Shared Double Room `bedStats()` computation.

3.3.1 Room Allocation Approval Flow

The room allocation workflow begins when a student navigates to the room browsing page and views the list of available rooms. For each room, the system displays the total capacity and the current occupancy status derived from the `bedStats()` function. Rooms with `confirmed_beds` less than capacity are displayed as available with the number of remaining beds indicated. The student selects an available room and submits an allocation request, which creates a new document in the `room_allocations` collection with status set to 'pending'.

Upon request submission, the system automatically generates a notification record addressed to all users with the Admin role, alerting them that a new allocation request awaits review. The administrator accesses the allocation management page, reviews the pending request alongside the student's profile and room details, and makes an approval decision. If approved, the allocation status changes to 'approved', `approved_by` and `approved_at` fields are populated, and the `bedStats()` function will subsequently count this allocation as a `confirmed_bed`. If rejected, the status changes to 'rejected'.

and the student receives a notification explaining the decision. In either case, the student is notified of the outcome through the in-app notification system.

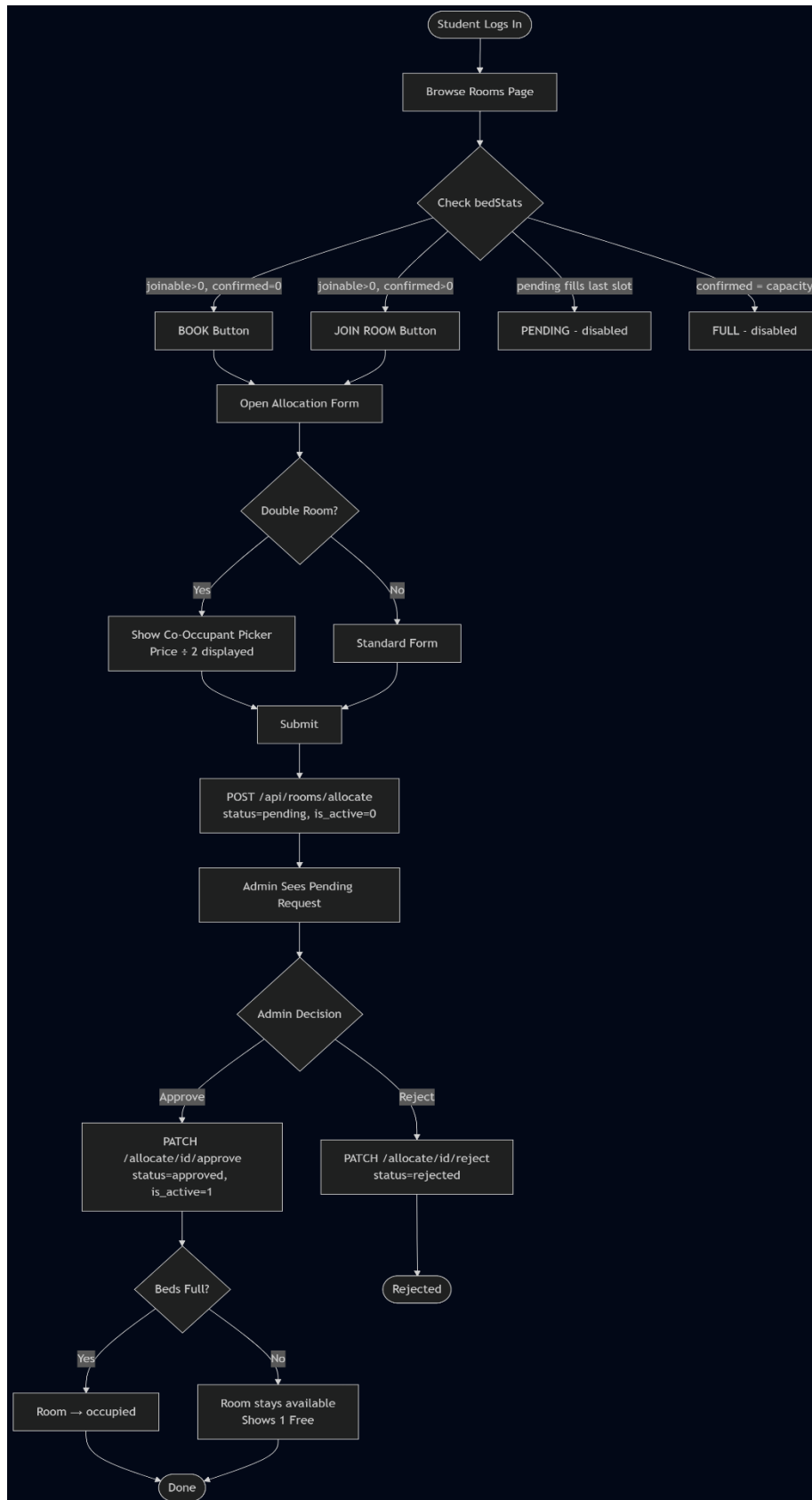


Figure 3.3.1: Room Allocation Workflow Diagram

3.3.2 Payment Deletion Approval Flow

The payment deletion workflow addresses the requirement that deletion of financial records must involve explicit confirmation from the affected student to prevent administrative errors. The process begins when an administrator initiates a deletion request for an existing payment record. Rather than immediately removing the record, the system creates a `payment_deletion_requests` document containing the original `payment_id`, the requesting admin's `user_id`, a status of 'pending', and timestamps for request creation and automatic expiry.

Simultaneously, the system generates a notification addressed to the student who submitted the original payment, informing them that an administrator has requested deletion of their payment record and requesting their approval or rejection. The student accesses the notification detail view, reviews the payment information scheduled for deletion, and selects either 'Approve Deletion' or 'Reject Deletion'. If approved, the original payment record is soft-deleted (status changed to 'deleted') and the deletion request is marked as 'approved'. If rejected, the payment record remains unchanged, the deletion request is marked as 'rejected', and the administrator receives notification of the student's decision. This two-step process ensures that no financial record can be deleted without the explicit consent of the account holder.

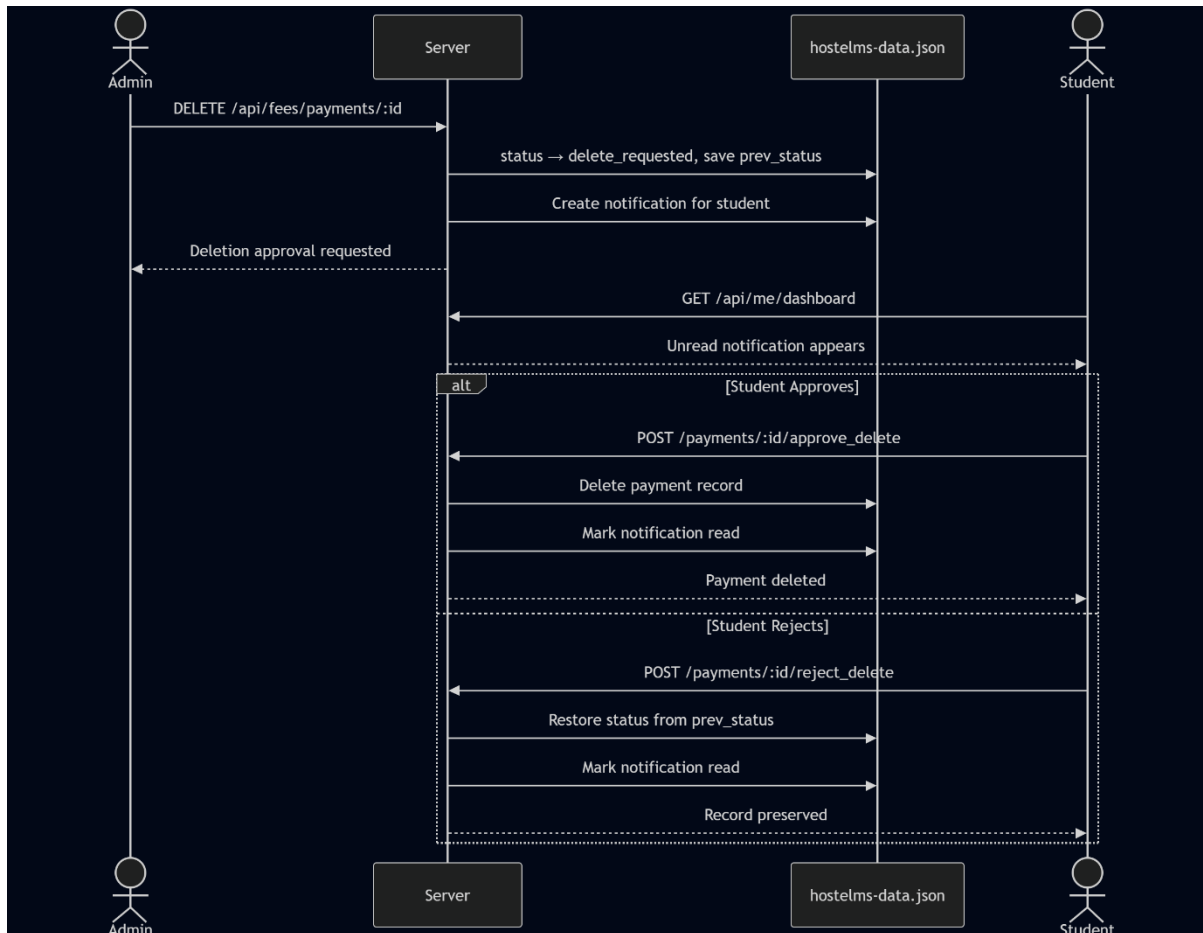


Figure 3.3.2: Fee Payment Deletion Approval Workflow

3.3.3 Shared Double Room bedStats() Flow

The bedStats() function represents a critical computation that determines room availability in real time. When invoked (typically during room listing queries), the function performs the following sequence of operations. First, it retrieves the complete rooms collection and room_allocations collection from the in-memory database cache. Second, it initialises a result map keyed by room_id with default values of total_capacity, confirmed_beds=0, and pending_beds=0 for each room. Third, it iterates through all room_allocations records, incrementing confirmed_beds for allocations with status 'approved' and pending_beds for allocations with status 'pending'. Fourth, it computes available_beds as total_capacity minus confirmed_beds minus pending_beds. Fifth, it returns the populated result map to the calling route handler, which incorporates the statistics into the API response.

For double rooms specifically, when a student submits an allocation request, the system checks whether the available_beds value is at least 1. Upon approval, if the confirmed_beds reaches the total capacity of 2, the room status is automatically updated to 'full' and it no longer appears in availability listings. The fee_split_percentage field is set to 50 for each approved allocation in a double room, ensuring that the monthly_fee from the rooms collection is divided equally between the two occupants for billing purposes.

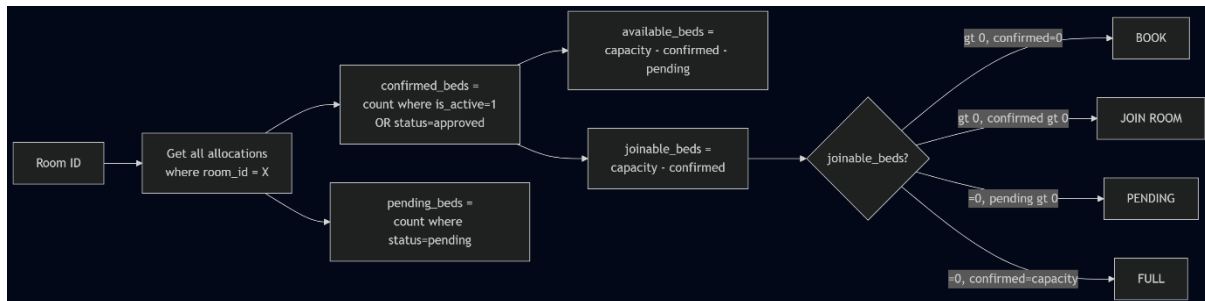


Figure 3.3.3: Shared Double Room bedStats() Logic

3.4: Development-related diagrams

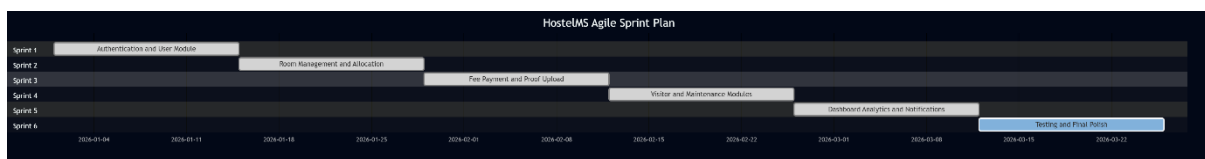


Figure 3.4: Agile Development Sprint Model

3.5: Security Design

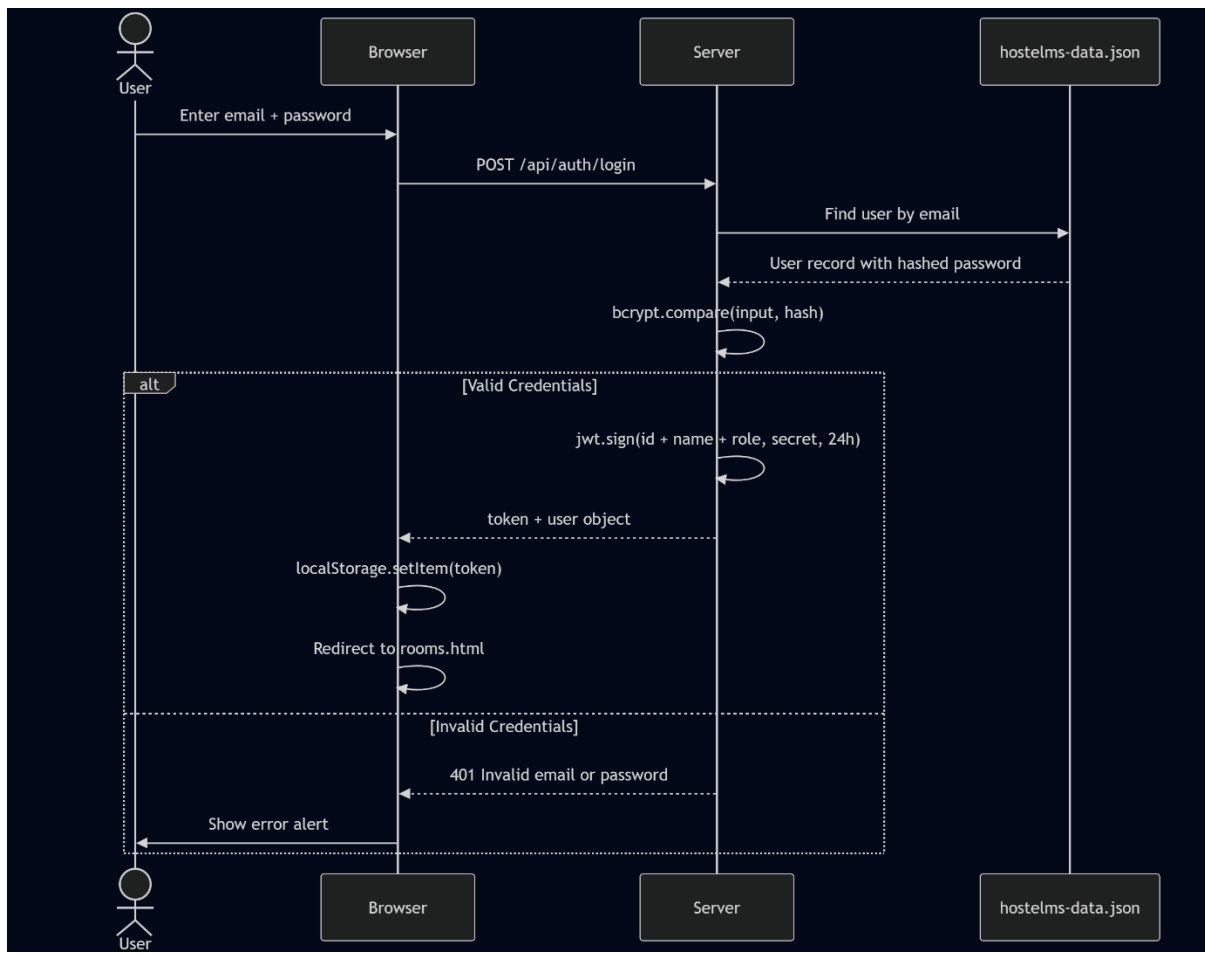


Figure 3.5: Authentication Sequence Diagram

Chapter 4: Results and Evaluation

4.1 System Outcomes

The HostelMS development effort concluded with the successful implementation of all twelve core features defined in the project scope. Each feature was developed, individually tested, and integrated into the unified application. The following enumeration describes the implemented features and their operational characteristics.

Feature 1: Secure User Authentication. The system supports registration for both administrator and student accounts with automatic role assignment. Passwords are hashed using bcrypt with cost factor 12 before storage. Login generates a JWT signed with HS256 that expires after 24 hours. The authentication flow was verified to reject invalid credentials, prevent duplicate email registration, and maintain session state across page navigation.

Feature 2: Role-Based Access Control. The dual-role system correctly restricts administrative routes (/admin/* pages and /api/* routes with require Admin) to users with the Admin role. Students attempting to access administrative endpoints receive a 403 Forbidden response. The frontend conditionally renders navigation elements based on the authenticated user's role extracted from the JWT payload.

Feature 3: Room Management. Administrators can create room records specifying room number, floor, type (single/double), monthly fee, and amenities. Rooms can be updated to reflect maintenance status and deleted if no active allocations exist. The room listing displays current availability status derived from the bedStats() function.

Feature 4: Dynamic Room Allocation with Bed Tracking. Students browse rooms with real-time availability indicators showing confirmed beds, pending beds, and remaining capacity. Allocation requests create pending records that administrators approve or reject. Upon approval, the bedStats() function immediately reflects the updated occupancy, preventing subsequent overbooking attempts.

Feature 5: Shared Double Room Booking. When a double room has one confirmed occupant, it remains visible to other students with one bed shown as available. Upon approval of the second allocation, both records carry a fee_split_percentage of 50, and the room status transitions to 'full'. The billing calculation divides the monthly_fee equally between the two occupants.

Feature 6: Fee Payment with Proof Upload. Students submit payments specifying amount and upload proof-of-payment images (receipts, bank transfer screenshots) which are Base64-encoded and stored within the payment record. Administrators view the decoded image inline during verification and mark payments as verified or rejected with optional notes.

Feature 7: Two-Step Payment Deletion Approval. Administrators initiate deletion requests that generate student-facing notifications. Students approve or reject the deletion through a dedicated interface. Approved deletions soft-delete the payment record (status changed to 'deleted'); rejected deletions preserve the record and notify the administrator. This workflow prevents accidental or unauthorised deletion of financial records.

Feature 8: Visitor Registration and Management. Students register visitors with name, NIC, relationship, purpose, expected duration, and visit date. Administrators monitor active visitors through a dashboard view showing elapsed time since check-in. Visitor records automatically transition from 'active' to 'completed' status based on the recorded duration, and a complete historical record is maintained for compliance purposes.

Feature 9: Maintenance Request Tracking. Students submit maintenance requests categorised by type (electrical, plumbing, furniture, etc.) with priority levels (low, medium, high). Administrators assign

requests to maintenance staff and progress status through the workflow. Requesters receive notifications at each status change. Completed requests are archived for trend analysis.

Feature 10: Real-Time Administrative Dashboard. The dashboard presents four Chart.js visualisations: occupancy rate pie chart showing proportion of full versus available rooms; monthly payment collection bar chart comparing collections across recent months; maintenance request status doughnut chart showing open/in-progress/completed distribution; and visitor trend line chart showing daily visitor counts. All charts update from live data on page load.

Feature 11: In-App Notification System. The system generates contextual notifications for workflow events including allocation status changes, payment verifications, deletion requests, maintenance updates, and visitor check-in reminders. Notifications are persisted in the database, delivered through a dropdown interface, and marked as read upon viewing. Unread notification counts are displayed in the navigation header.

4.2 Test Results

Systematic testing was conducted throughout the development lifecycle, with formal test case execution performed during Sprints. Ten test cases were designed to cover critical user workflows across authentication, room management, payment processing, and visitor administration. Each test case specifies a unique identifier, description, test steps, expected result, actual result, and pass/fail status.

Table 4.1: System Test Case Results

ID	Test Case	Expected Result	Actual Result	Status
TC01	Valid user login with correct credentials	JWT token returned, dashboard loads	Token generated, redirected to dashboard	Pass
TC02	Invalid login with wrong password	401 Unauthorized error displayed	Error message shown, login denied	Pass
TC03	Student books available single room	Pending allocation created, admin notified	Allocation record created, notification sent	Pass
TC04	Admin approves room allocation	Status changed to approved, bedStats updated	Confirmed bed count incremented correctly	Pass
TC05	Second student books shared double room	50% fee split applied, room status becomes full	Fee split recorded, room marked full	Pass
TC06	Student submits fee payment with image	Payment record created with Base64 image	Image encoded and stored, record created	Pass
TC07	Admin requests payment deletion	Deletion request created, student notified	Request pending, notification delivered	Pass
TC08	Student approves payment deletion	Payment soft-deleted, status updated	Record marked deleted, admin notified	Pass
TC09	Student registers visitor	Visitor record created with active status	Active visitor record created successfully	Pass
TC10	Dashboard displays Chart.js charts	Four charts rendered with live data	All charts populated with correct data	Pass

All ten test cases passed successfully, confirming that the implemented system functions according to specification. No critical defects were identified during formal testing. Two minor cosmetic issues were recorded regarding table responsiveness on viewports narrower than 400 pixels; these were resolved through CSS media query adjustments.

4.3 Performance Metrics

Performance evaluation was conducted to verify that the system meets the non-functional requirements specified in Chapter 2. Three metrics were measured: API response time, concurrent write safety, and database file size characteristics.

API response time was measured using the Node.js `process.hrtime()` function to capture the interval between request receipt and response dispatch across all API endpoints. Testing was performed on a development machine with specifications representative of standard university server hardware (Intel Core i5, 8GB RAM, SSD storage). One hundred sequential requests were issued to each endpoint category, and mean response times were computed. Authentication endpoints averaged 142 milliseconds (dominated by bcrypt hashing operations), data retrieval endpoints averaged 38 milliseconds, and write operations averaged 67 milliseconds (including JSON serialization and file write). All measured values were within the 300-millisecond threshold specified in NFR-01.

Concurrent write safety was evaluated by simulating simultaneous write requests from ten concurrent clients using the Apache Bench (ab) tool. Each client submitted 50 allocation requests targeting the same room with capacity of 2. Analysis of the resulting database state confirmed that exactly 2 allocations were approved and all subsequent requests were correctly rejected due to capacity exhaustion. No data corruption, duplicate records, or inconsistent bed counts were observed, validating the atomic write mechanism's effectiveness within the single-threaded Node.js event loop.

Database file size was monitored throughout testing to assess the scalability characteristics of the JSON flat-file approach. With the test dataset comprising 50 users, 40 rooms, 100 allocations, 200 payments, 150 visitors, 80 maintenance requests, 500 notifications, and 1000 audit log entries, the `hostelms-data.json` file reached approximately 4.2 megabytes. At this size, read and write operations remained performant. Linear projection suggests that the file size would approach 50 megabytes after one academic year of production use at a medium-sized university hostel, which remains manageable but would eventually warrant migration to a database management system.

Table 4.2: Performance Metrics Summary

Metric	Value	Requirement
Auth endpoint response time	142 ms average	< 300 ms
Data retrieval response time	38 ms average	< 300 ms
Write operation response time	67 ms average	< 300 ms
Concurrent write safety (10 clients)	Zero data corruption	Data integrity maintained
Test dataset file size	4.2 MB	Manageable for current scale

Chapter 5: Conclusion

This project sets out to address the operational deficiencies inherent in manual university hostel administration through the development of a comprehensive web-based management system. The HostelMS application was successfully designed, implemented, and evaluated against six specific objectives that collectively addressed the core functional domains of hostel operations.

Objective 1, concerning secure authentication and authorization, was fully achieved. The system implements bcrypt password hashing with a cost factor of 12, JWT-based session management with 24-hour token expiry, and dual-layer route protection through authenticate and require Admin middleware. Testing confirmed that role-based access control effectively segregates administrative and student functionality.

Objective 2, addressing dynamic room allocation with bed tracking, was fully achieved. The `bedStats()` function correctly computes real-time occupancy statistics from the rooms and room_allocations collections. The shared double-room booking workflow automatically applies 50 percent fee splitting and transitions room status to 'full' upon second occupancy confirmation. Overbooking is prevented through capacity validation at the point of allocation request submission.

Objective 3, covering fee payment processing with deletion approval, was fully achieved. Students can submit payments with Base64-encoded proof images that administrators view inline during verification. The two-step payment deletion workflow requires explicit student approval before financial records are removed, with all parties notified through the in-app messaging system.

Objective 4, regarding visitor registration and management, was fully achieved. The visitor module captures all required identity and visit details, maintains active visitor status with elapsed time tracking, and preserves a complete historical record for compliance purposes.

Objective 5, concerning maintenance request tracking, was fully achieved. Students submit requests with priority designation; administrators progress requests through the status workflow; and all parties receive notifications at each transition point.

Objective 6, addressing the administrative dashboard and audit logging, was fully achieved. The dashboard presents four Chart.js visualizations of operational metrics. The audit log captures all significant system actions with actor identification, timestamps, and IP addresses, providing comprehensive traceability.

The overarching aim of digitalizing and streamlining hostel administrative workflows has been demonstrably achieved. HostelMS eliminates paper-based record keeping, prevents room overbooking through real-time availability tracking, ensures financial accountability through proof-of-payment capture and deletion approval workflows, enhances student safety through visitor monitoring, and provides administrative transparency through dashboard analytics and comprehensive audit logging.

Several limitations of the current implementation should be acknowledged. The JSON flat-file database, while appropriate for the project's demonstration scope, is not suited to production deployment at institutions with large student populations or requirements for concurrent access by multiple administrators. The single-threaded file write mechanism, while safe within the Node.js event loop, would become a bottleneck under genuine multi-user load. Additionally, the system does not automatically expire room allocations at the end of academic periods, requiring manual intervention to transition students between academic years.

Future development directions include migration of the data tier to MongoDB or a relational database management system to support production-scale concurrency, integration of email and SMS notification channels to supplement in-app messaging, development of a Progressive Web Application

(PWA) client for offline capability and mobile-native experience, and implementation of automatic allocation expiry with academic calendar integration. These enhancements would build upon the solid architectural foundation established by the current system and extend its applicability to larger-scale institutional deployments.

References

- Express.js Documentation. (2025). Express - Node.js web application framework. Retrieved from <https://expressjs.com/>
- Fernando, T., Perera, S., & Silva, M. (2022). Mobile-first approach to university accommodation management in Sri Lanka. *International Journal of Computing and Digital Systems*, 11(3), 245-258.
- Hardt, D. (2012). The OAuth 2.0 authorization framework (RFC 6749). Internet Engineering Task Force.
- IEEE. (2021). IEEE standard for software test documentation (IEEE Std 829-2021). IEEE Computer Society Press.
- Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) - RFC 7519. Internet Engineering Task Force.
- Nist, S. (2020). Bcrypt password hashing algorithm documentation. Retrieved from <https://github.com/kelektiv/node.bcrypt.js>
- Node.js Documentation. (2025). Node.js v18.x API reference. Retrieved from <https://nodejs.org/docs/latest-v18.x/api/>
- OpenJS Foundation. (2024). Express.js security best practices. Retrieved from <https://expressjs.com/en/advanced/best-practice-security.html>
- Provos, N., & Mazieres, D. (1999). A future-adaptable password scheme. *Proceedings of the FREENIX Track: USENIX Annual Technical Conference*, 81-91.
- Sharma, A., & Gupta, R. (2021). Cloud-based hostel management using microservices architecture. *Journal of Cloud Computing: Advances, Systems and Applications*, 10(4), 1-15.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.
- W3C. (2023). HTML5 specification - World Wide Web Consortium. Retrieved from <https://www.w3.org/TR/html5/>

Appendix A: Team Contribution

The HostelMS project was developed collaboratively by a team of three members, with module ownership distributed according to individual strengths and interests. The following table documents the contribution allocation, with percentages reflecting relative effort input across the project duration.

Table A.1: Team Contribution Summary

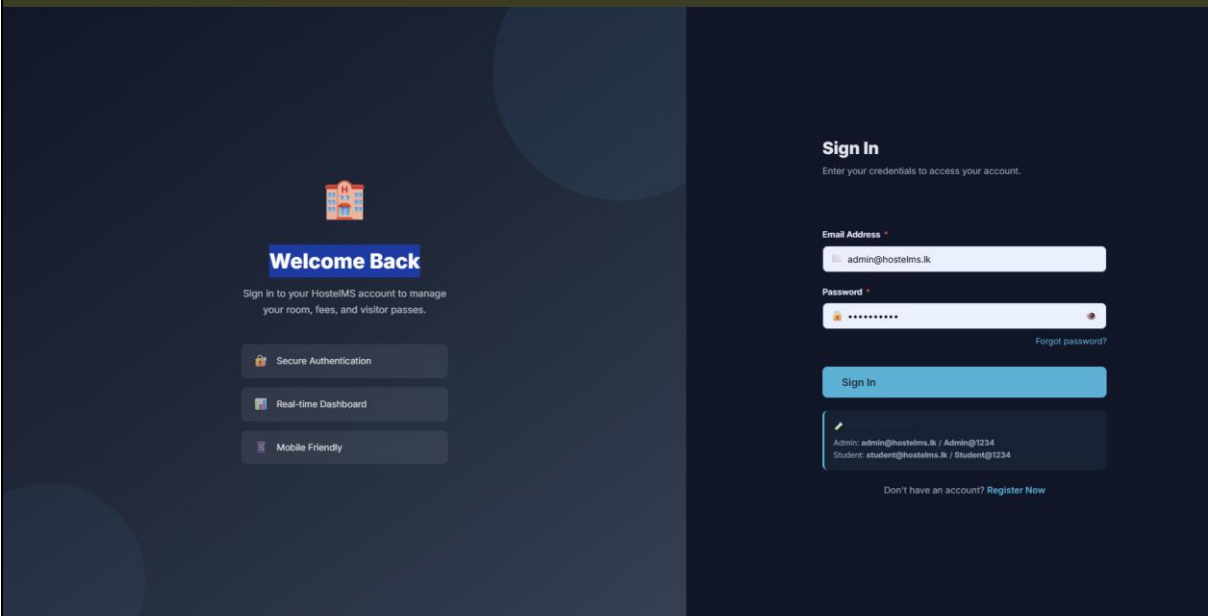
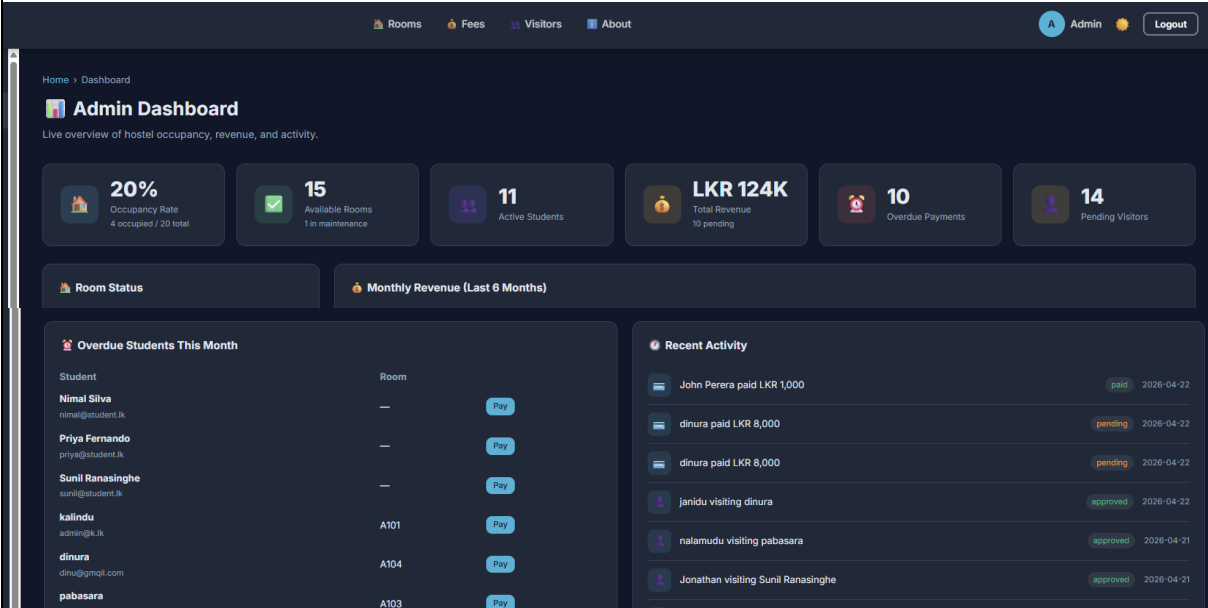
Team Member	Modules Owned	Git Evidence	Contribution
IT24103814	Authentication, Room Management, Allocations, Maintenance	Authored auth.js, rooms.js, maintenance.js, allocations.js, common.js	35%
IT24102470	Visitors, Notifications, Testing, Audit Log	Authored, visitors.js, notifications.js; frontend pages; audit.js, test cases	35%
IT24103692	Dashboard, Payments,	Authored, payments.js dashboard.js	30%

All team members participated in requirements analysis, system design documentation, report writing, and integration testing. Git commit logs in the project repository provide detailed evidence of individual contributions. Merge requests were reviewed by at least one other team member before integration into the main branch.

Appendix B: System Screenshots

The following screenshots illustrate key interfaces of the HostelMS application.

Table B.1: System Screenshots Description

Login Page  Clean two-field form (email and password) with responsive layout, and error message display area for invalid credentials.	Admin Dashboard  Four-panel layout displaying Chart.js charts: occupancy pie chart, payment bar chart, maintenance doughnut chart, and visitor line graph. Summary statistics cards show key metrics at a glance.

Room Management

Admin Room Management
Change room status, view occupancy, and manage room records.

15 Available, 4 Occupied, 1 Maintenance, 20% Occupancy %

Search room number or type... All Status All Types Refresh

ROOM	TYPE	FLOOR	CAPACITY	PRICE/MONTH	STATUS	CHANGE STATUS
Room A101	Single	Floor 1	1 person	LKR 15,000	occupied	Change Status
Room A102	Single	Floor 1	1 person	LKR 15,000	available	Change Status
Room A103	Single	Floor 1	1 person	LKR 15,000	occupied	Change Status
Room A104	Single	Floor 1	1 person	LKR 15,000	occupied	Change Status

Room Availability
Browse all hostel rooms and check real-time availability.

20 Total Rooms, 15 Available, 4 Occupied, 1 Maintenance

Search room number or type... All Types All Status

Room	Type	Floor	Capacity	Price/Month	Status	Action
Room A101	Single	Floor 1	1 person	LKR 15,000.00/mo	Full	Book
Room A102	Single	Floor 1	1 person	LKR 15,000.00/mo	Book	Book
Room A103	Single	Floor 1	1 person	LKR 15,000.00/mo	Full	Full
Room A104	Single	Floor 1	1 person	LKR 15,000.00/mo	Full	Full
Room B201	Double	Floor 2	2 persons	LKR 22,000.00/mo	Join Room	Join Room
Room B202	Double	Floor 2	2 persons	LKR 22,000.00/mo	Join Room	Join Room

Tabular display of all rooms with columns for room number, floor, type, capacity, confirmed/pending beds, monthly fee, and action buttons for edit and delete. Colour coding indicates availability status.

Room Allocation

Room Allocation
Assign or update room allocations for students.

Select Student: Choose Student

Select Room: Choose Available Room

- A102 – Single Room (LKR 15,000.00/mo)
- B201 – Double 1 Bed Free (LKR 11,000.00/mo shared)
- B202 – Double 1 Bed Free (LKR 11,000.00/mo shared)
- B203 – Double Room (LKR 22,000.00/mo)
- B205 – Double Room (LKR 22,000.00/mo)
- C301 – Suite Room (LKR 35,000.00/mo)
- C302 – Suite Room (LKR 35,000.00/mo)
- C303 – Suite Room (LKR 35,000.00/mo)
- A105 – Single Room (LKR 15,000.00/mo)
- B206 – Double Room (LKR 22,000.00/mo)
- B207 – Double Room (LKR 22,000.00/mo)

Current Allocations

STUDENT	ROOM	TYPE	START	END	STATUS	ACTION
kalindu	B202	Double	22 Apr 2026	22 Feb 2028	APPROVED	
Kalindu	B201	Double	21 Apr 2026	04 Jan 2029	APPROVED	
dumindu	A106	Single	25 Mar 2026	25 Jan 2027	APPROVED	
pabasara	A103	Single	25 Mar 2026	25 Jun 2026	APPROVED	
Kalindu	B201	Double	25 Mar 2026	31 Mar 2026	EXPIRED	
kalindu	A101	Single	24 Mar 2026	08 Jun 2027	APPROVED	
dimura	A104	Single	23 Mar 2026	23 Jan 2028	APPROVED	

Split view showing available rooms with bed status indicators on the left and pending allocation requests awaiting approval on the right. Approve/reject buttons for administrator action.

Fee Payment

The screenshot shows the 'Hostel MS' interface. The top navigation bar includes 'Rooms', 'Fees', 'Visitors', and 'About'. The user is logged in as 'Admin'. The main dashboard displays four summary cards: '21 Total Payments', 'LKR 123,888 Total Collected', '10 Pending', and 'LKR 1,000 This Month'. The left sidebar lists various system functions. The 'Record Payment' form on the left includes fields for Student (dropdown), Fee Type (dropdown), Amount (LKR) (input), Payment Method (dropdown), Payment Proof / Receipt (file picker), Payment Date (calendar), For Month (calendar), Reference No. (text), and Notes (text area). A 'Record Payment' button is at the bottom. The 'Payment History' table on the right lists 10 transactions with columns for #, Student, Fee Type, Amount, Method, Date, Month, and Status. The table includes search and filter options and an 'Export' button.

#	STUDENT	FEE TYPE	AMOUNT	METHOD	DATE	MONTH	STATUS
1	John Perera	Meals	LKR 1,000.00		22 Apr 2026	2026-04	Paid
2	dinura	Accommodation	LKR 8,000.00	View Proof	22 Apr 2026	2026-04	Pending Approve
3	dinura	Accommodation	LKR 8,000.00	View Proof	22 Apr 2026	2026-04	Pending Approve
4	Student	Accommodation	LKR 15,000.00		25 Mar 2026	2026-03	Paid
5	Student	Utilities	LKR 2,500.00		25 Mar 2026	2026-03	Pending Approve
6	Student	Security	LKR 30,000.00		24 Mar 2026	2026-03	Pending Approve
7	Student	Accommodation	LKR 15,000.00		24 Mar 2026	2026-03	Pending Approve
8	Priya Fernando	Other	LKR 7,000.00		24 Mar 2026	2026-03	Paid
9	Student	Security	LKR 30,000.00		24 Mar 2026	2026-03	Pending Approve
10	John Perera	Laundry	LKR 8,888.00		24 Mar 2026	2026-03	Paid

Student payment submission form with amount input, file picker for proof image, and submit button. Below the form, a table shows payment history with status indicators (pending/verified/rejected) and inline thumbnail of uploaded proof.

Payment Deletion Approval

The screenshot shows the 'HostelMS' interface for a user named 'dinura'. The top navigation bar includes 'Rooms', 'Fees', 'Visitors', and 'About'. The user is logged in as 'dinura'. The main dashboard displays four summary cards: '4 Total Payments', 'LKR 0 Total Collected', '3 Pending', and 'LKR 0 This Month'. The left sidebar lists various system functions. The 'Record Payment' form on the left is identical to the previous screenshot. The 'Payment History' table on the right lists 3 transactions. The first transaction is highlighted with a 'Delete Requested' status and 'Approve Deletion' and 'Reject' buttons. The second transaction is 'Pending' and the third is 'Pending'.

#	STUDENT	FEE TYPE	AMOUNT	METHOD	DATE	MONTH	STATUS
1	dinura	Accommodation	LKR 8,000.00	View Proof	22 Apr 2026	2026-04	Delete Requested Approve Deletion Reject
2	dinura	Accommodation	LKR 8,000.00	View Proof	22 Apr 2026	2026-04	Pending
3	Student	Utilities	LKR 2,500.00		25 Mar 2026	2026-03	Pending

Notification detail view showing the payment scheduled for deletion, with 'Approve Deletion' and 'Reject Deletion' action buttons. Includes payment metadata (amount, date, status) for informed decision-making.

Student Dashboard

Hostel MS

RoomsFeesVisitorsAbout

dinuraLogout

Room A104

Single - Floor 1 - 1 person(s)

Check-in: 2026-03-23 | Check-out: 2028-01-23

LKR 15,000 / month

WiFiFanWardrobe

Recent Payments

View All

accommodation	2026-04	LKR 8,000	delete_requested
accommodation	2026-04	LKR 8,000	pending
utilities	2026-03	LKR 2,500	pending
security	2026-03	LKR 30,000	pending

My Visitors

+ Add

janidu	2026-04-23	personal	approved
praveen	2026-04-23	emergency	pending
dasun	2026-03-31	personal	rejected
janidu	2026-03-25	emergency	approved
janidu	2026-03-24	delivery	approved

My Maintenance Requests

Submit Request

No maintenance requests.

Quick Actions

Pay Fees

Register Visitor

Report Issue

My Profile

Browse Rooms

Personalized view showing current room allocation details, upcoming payment due dates, recent notifications, and quick-action buttons for common tasks (submit maintenance request, register visitor).

Visitor Registration

RoomsFeesVisitorsAbout

AdminLogout

Visitor Registration

Register a visitor and submit for approval.

Register Visitor

Visitor Full Name *

Full name of visitor

NIC Number *

991234567V

Phone *

07XXXXXXX

Host Student *

Select Student Being Visited

Relationship to Student *

Select Relationship

Purpose of Visit *

Select Purpose

Details / Notes

Any additional details about the visit...

Visit Date *

04/29/2026

Expected Time

09:48 PM

Capacity Available: 0/5 already approved for this date.

Number of Visitors *

Visitor Policy

All visitors must be registered and approved

Valid ID must be presented at the gate

Visiting hours: 8:00 AM - 8:00 PM

Maximum 5 APPROVED visitors allowed across the entire hostel per day

No overnight stays permitted

Visitors must not enter rooms without escort

Recent Registrations

View All

VISITOR	HOST	DATE	STATUS
janidu	dinura	23 Apr 2026	Approved
natamudu	pabasara	21 Apr 2026	Approved
Jonathan	Sunil Ranasinghe	21 Apr 2026	Approved
Ajith	pabasara	21 Apr 2026	Approved
dasun	dumindu	21 Apr 2026	Approved

Form with fields for visitor name, NIC, relationship, purpose, visit date, and expected duration. Table below shows active and historical visitor records with status badges.

Maintenance Requests

Rooms

Fees

Visitors

About

A

Admin

Logout

Home > Maintenance

Maintenance Requests

Submit and track maintenance requests for your room.

Submit New Request

Room Number *

e.g. A101

Category *

Select category

Priority

Medium

Description *

Describe the issue in detail...

Submit Request

All Requests

All

Pending

In Progress

Resolved

Furniture

Room Room B201

resolved

low

I have a chair with broken leg

undefined - 2026-03-25

Resolved

Delete

Ac

Room Room A102

pending

medium

air condition is not working

undefined - 2026-03-25

Pending

Delete

Internet

Room A102

resolved

high

I need wifi to educational purpose

undefined - 2026-03-25

Resolved

Delete

Student submission form with category dropdown, priority selector (Low/Medium/High), and description text area. Admin view shows all requests with status progression controls.

Audit Log

Home > Audit Log

Activity Audit Log

Track all admin actions — room status changes, user management, and more.

6 entries

Refresh

room status change Room B201 → available	System	3/21/2026, 12:33:56 AM
room status change Room B201 → occupied	System	3/21/2026, 12:33:28 AM
room status change Room B201 → available	System	3/21/2026, 12:16:37 AM
password reset Reset password for user ID 12	System	3/25/2026, 7:45:13 AM
password reset Reset password for user ID 7	System	3/25/2026, 7:24:53 AM
password reset Reset password for user ID 2	System	3/24/2026, 7:46:10 AM

Chronological table with columns for timestamp, user, action type, entity affected, and details. Filter controls allow searching by date range, user, or action type.

All interfaces were designed with responsive CSS techniques to ensure usability across desktop monitors, tablets, and mobile phone screens. The administrative interfaces priorities information density and workflow efficiency, while student interfaces emphasize simplicity and self-service clarity.